

Phát triển ứng dụng web

Responsive Design

Nội dung

- ☐ Nguyên lý thiết kế web hiện đại
- ☐ CSS – Layout
- ☐ Responsive web design

Nội dung

- ❑ Nguyên lý thiết kế web hiện đại
- ❑ CSS – Layout
- ❑ Responsive web design

Tại sao Thiết kế lại quan trọng?



UI vs. UX

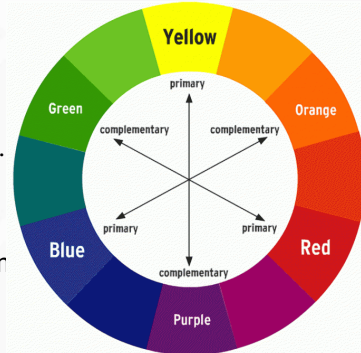
- ❑ **UI** (User Interface - Giao diện người dùng): Các yếu tố người dùng nhìn thấy và tương tác (nút bấm, màu sắc, layout, icon).
 - ❑ **Mục tiêu:** Thẩm mỹ, hấp dẫn, nhất quán.
- ❑ **UX** (User Experience - Trải nghiệm người dùng): Cảm nhận, cảm xúc của người dùng khi sử dụng sản phẩm. Nó có dễ dùng không? Có giải quyết được vấn đề của họ không
 - ❑ **Mục tiêu:** Hữu ích, dễ sử dụng, hiệu quả.

7 Nguyên lý Vàng trong Thiết kế Thị giác



Lý thuyết màu sắc

- ☐ Vòng tròn màu sắc
- ☐ Tâm lý học màu sắc
 - ☐ **Xanh dương:** Tin tưởng, chuyên nghiệp.
 - ☐ **Đỏ:** Năng lượng, khẩn cấp, nguy hiểm.
 - ☐ **Xanh lá:** Tăng trưởng, tự nhiên, bình yên
- ☐ Quy tắc 60-30-10:
 - ☐ **60%** Màu chủ đạo (thường là màu nền).
 - ☐ **30%** Màu thứ cấp (làm nổi bật các khu vực).
 - ☐ **10%** Màu nhấn (dành cho các nút kêu gọi hành động - CTA).



Typography

- ☐ Serif vs. Sans-serif:
 - ☐ **Serif (chữ có chân):** Cổ điển, trang trọng, dễ đọc trong văn bản in. (Times New Roman).
 - ☐ **Sans-serif (chữ không chân):** Hiện đại, sạch sẽ, dễ đọc trên màn hình kỹ thuật số. (Arial, Helvetica, Roboto).
- ☐ Các quy tắc vàng:
 - ☐ **Giới hạn số lượng font:** Sử dụng tối đa 2-3 font trong một trang web.
 - ☐ **Đảm bảo độ tương phản:** Màu chữ và màu nền phải đủ tương phản.
 - ☐ **Kích thước và khoảng cách:** Chú ý đến **font-size**, **line-height** (chiều cao dòng, thường là 1.5 lần **font-size**) và **letter-spacing** để dễ đọc.

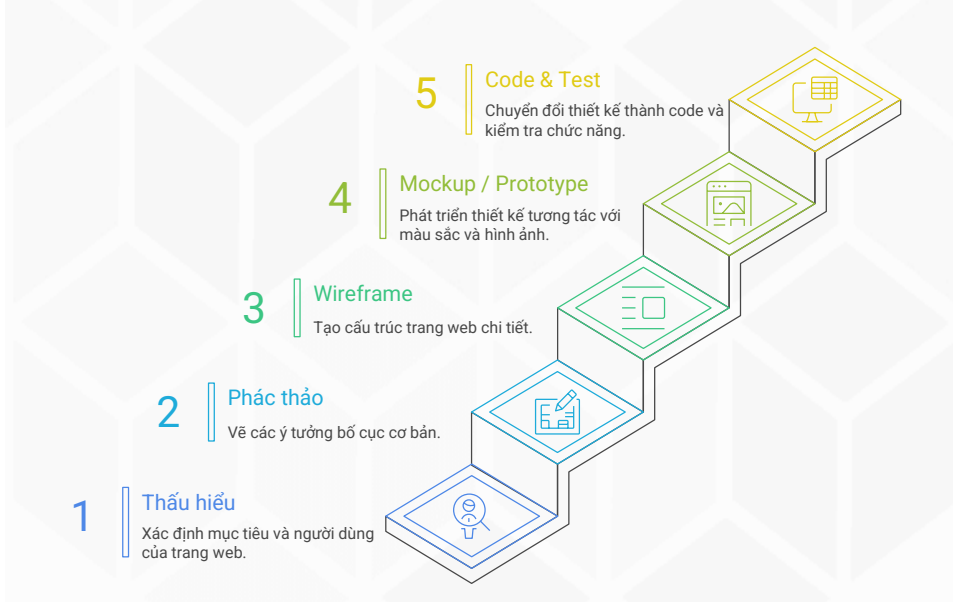
Responsive Design

- ☐ **Khái niệm:** Là phương pháp thiết kế giúp trang web hiển thị tốt trên mọi kích thước màn hình (Desktop, Tablet, Mobile).
 - ☐ Hiện nay hơn 50% lưu lượng truy cập web đến từ thiết bị di động.
- ☐ **Triết lý "Mobile-First":** Thiết kế cho màn hình di động trước, sau đó mới mở rộng lên các màn hình lớn hơn.

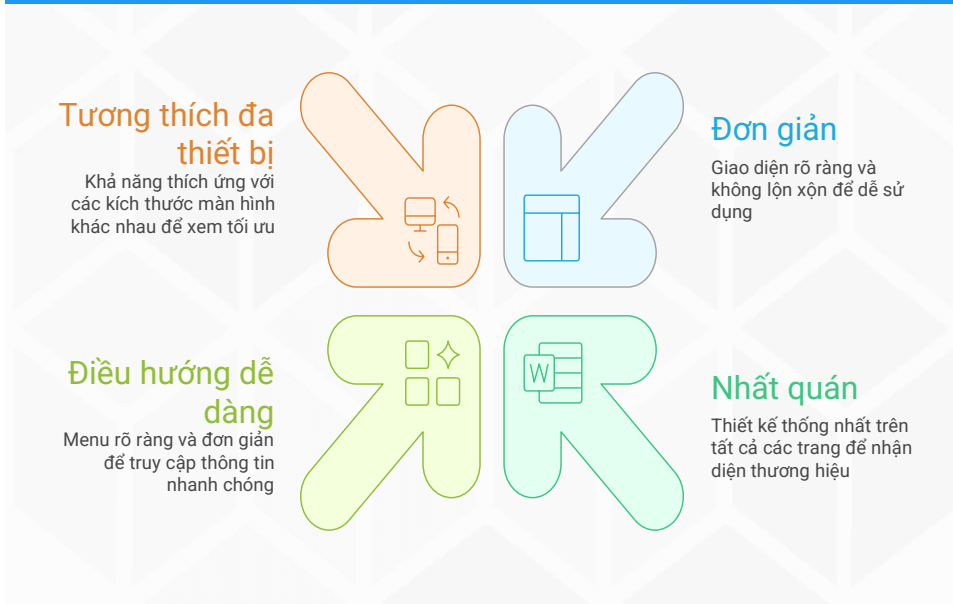
Accessibility

- ☐ **Khái niệm:** Thiết kế web để **mọi người**, bao gồm cả người khuyết tật, đều có thể sử dụng được. Đây không phải là một tính năng, mà là một **yêu cầu cơ bản**.
- ☐ Ví dụ:
 - ☐ **Thẻ alt cho hình ảnh:** Cung cấp mô tả cho trình đọc màn hình.
 - ☐ **Độ tương phản màu sắc:** Đảm bảo người có thị lực kém vẫn đọc được nội dung.
 - ☐ **Sử dụng HTML ngữ nghĩa:** (`<nav>`, `<main>`, `<button>`) giúp các công nghệ hỗ trợ hiểu cấu trúc trang.
 - ☐ Hỗ trợ điều hướng bằng bàn phím.

Quy trình thiết kế cơ bản



Nguyên lý cơ bản của thiết kế web chuyên nghiệp



Nội dung

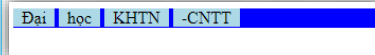
- ☐ Nguyên lý thiết kế web hiện đại
- ☐ **CSS – Layout**
- ☐ Responsive web design

Nội dung

- ☐ **Display**
- ☐ Flexible box layout
- ☐ Grid layout
- ☐ CSS – Page Layout

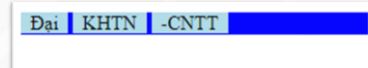
Block – Inline (default)

```
<html>
  <head>
    <style>
      div {
        background-color: blue;
      }
      span {
        background-color: lightblue;
        padding: 0 0.5rem;
      }
    </style>
  </head>
  <body>
    <div>
      <span>Đại</span>
      <span>học</span>
      <span>KHTN</span>
      <span>-CNTT</span>
    </div>
  </body>
</html>
```



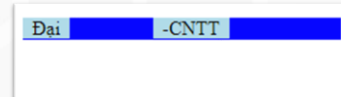
Display: none

```
<head>
  <style>
    div {
      background-color: blue;
    }
    span {
      background-color: lightblue;
      padding: 0 0.5rem;
    }
    .dnone {
      display: none;
    }
  </style>
</head>
<body>
  <div>
    <span>Đại</span>
    <span class="dnone">học</span>
    <span>KHTN</span>
    <span>-CNTT</span>
  </div>
</body>
```



Visibility: hidden

```
<head>
  <style>
    div {
      background-color: blue;
    }
    span {
      background-color: lightblue;
      padding: 0 0.5rem;
    }
    .dnone {
      display: none;
    }
    .vhidden {
      visibility: hidden;
    }
  </style>
</head>
<body>
  <div>
    <span>Đại</span>
    <span class="dnone">học</span>
    <span class="vhidden">KHTN</span>
    <span>-CNTT</span>
  </div>
</body>
```



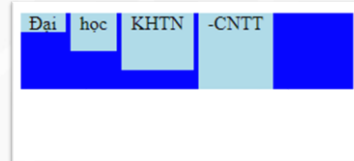
Display: block

```
<head>
  <style>
    div {
      background-color: blue;
    }
    span {
      background-color: lightblue;
      padding: 0 0.5rem;
      display: block;
    }
  </style>
</head>
<body>
  <div>
    <span>Đại</span>
    <span>học</span>
    <span>KHTN</span>
    <span>-CNTT</span>
  </div>
</body>
```



Display: inline-block

```
<style>
div {
  background-color: blue;
}
span {
  background-color: lightblue;
  padding: 0 0.5rem;
  display: inline-block;
}
span:nth-child(1) {
  height: 1rem;
}
span:nth-child(2) {
  height: 2rem;
}
span:nth-child(3) {
  height: 3rem;
}
span:nth-child(4) {
  height: 4rem;
}
</style>
```



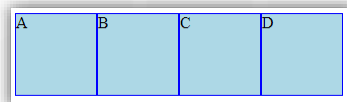
Nội dung

- ☐ *Display*
- ☐ **Flexible box layout**
- ☐ Grid layout
- ☐ CSS – Page Layout

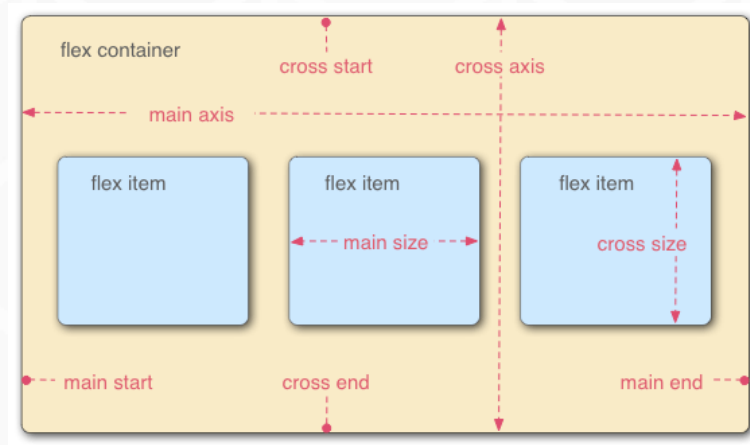
Flexible box layout (flexbox)

- ❑ Giúp xây dựng bố cục linh hoạt, đáp ứng với nhiều độ phân giải. Nó cung cấp tổng thể khả năng điều chỉnh kích thước của các thành phần container để hiển thị theo cách hiệu quả và dễ đọc nhất có thể.

```
<head>
  <style>
    .container {
      display: flex;
    }
    .item {
      width: 5rem;
      height: 5rem;
      background-color: lightblue;
      border: solid 1px blue;
    }
  </style>
</head>
<body>
  <div class="container">
    <div class="item">A</div>
    <div class="item">B</div>
    <div class="item">C</div>
    <div class="item">D</div>
  </div>
</body>
```



Flex Model



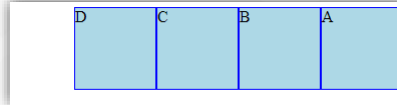
Flex Direction

- ❑ **Flex direction** xác định chiều nào và hướng nào các thành phần bên trong sẽ được sắp đặt. Có 2 tùy chọn cho **chiều** là theo hàng hay cột và 2 tùy chọn cho **hướng** là bình thường hoặc đảo ngược.

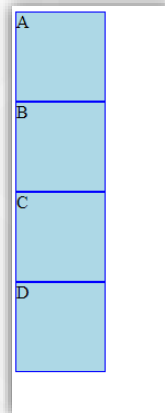
- ☐ row
- ☐ row-reverse
- ☐ column
- ☐ column-reverse

Flex Direction

```
.container {  
  display: flex;  
  flex-direction: row-reverse;  
}
```



```
.container {  
  display: flex;  
  flex-direction: column;  
}
```



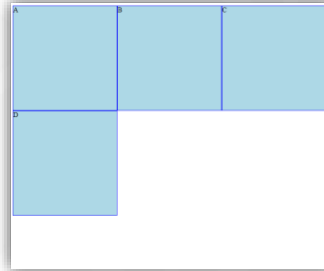
Flex Wrap

- ❑ Mặc định thì **container** sẽ cố gắng sắp xếp các thành phần con trên cùng trục chính (**main-axis**). Để thay đổi thì có thể sử dụng **flex-wrap** với các giá trị phù hợp:

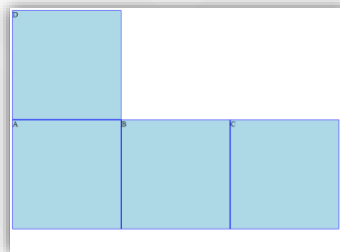
- ❑ wrap
- ❑ no-wrap
- ❑ wrap-reverse

Flex wrap

```
.container {  
  display: flex;  
  flex-wrap: wrap;  
}
```



```
.container {  
  display: flex;  
  flex-wrap: wrap-reverse;  
}
```



Flex flow

- ❑ **flex-flow** là property giúp thiết lập đồng thời cả **flex-direction** và **flex-wrap**

```
.container {  
  display: flex;  
  flex-flow: row wrap;  
}
```



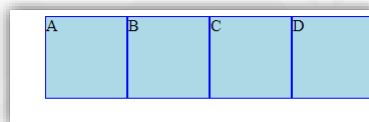
```
.container {  
  display: flex;  
  flex-direction: row;  
  flex-wrap: wrap;  
}
```

Justify Content

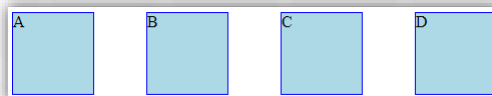
❑ ***justify-content*** định nghĩa cách xếp đặt các thành phần theo trục chính (***main-axis***). Có 6 cách thiết lập khác nhau:

- ❑ flex-start
- ❑ flex-end
- ❑ center
- ❑ space-between
- ❑ space-around
- ❑ space-evenly

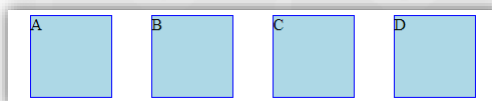
```
.container {  
  display: flex;  
  justify-content: flex-end;  
}
```



space-between



space-around

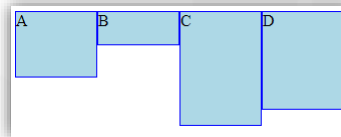


Align Items

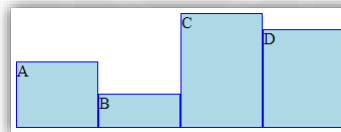
❑ ***align-items*** xác định cách các ***item*** được định vị trong trục phụ (***cross-axis***). Nó tương tự như ***justify-content*** nhưng trục giao với dòng nó được căn chỉnh. Các giá trị có thể thiết lập:

- ❑ flex-start
- ❑ flex-end
- ❑ center
- ❑ stretch
- ❑ baseline

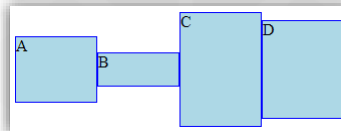
```
.container {  
  display: flex;  
  align-items: flex-start;  
}
```



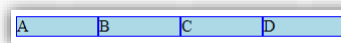
flex-end



center



stretch

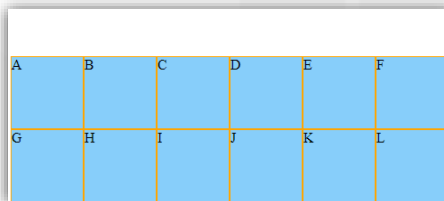


Align Content

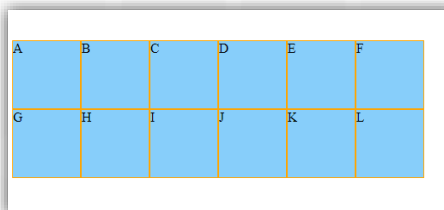
❑ ***align-content*** xác định khoảng cách giữa các ***line*** nếu chúng được ***wrap***. Các giá trị có thể thiết lập:

- ❑ flex-start
- ❑ flex-end
- ❑ center
- ❑ stretch
- ❑ space-between
- ❑ space-around

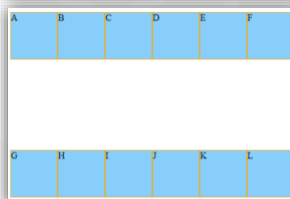
```
.container {  
  display: flex;  
  align-content: flex-end;  
  height: 20rem;  
  flex-wrap: wrap;  
}
```



center



space-between



Order

```
.a {  
  order: 1;  
}  
.b {  
  order: -2;  
}
```

```
<div class="item a">A</div>  
<div class="item b">B</div>  
<div class="item">C</div>  
<div class="item">D</div>
```



Flex Grow

```
.item {  
  width: 5rem;  
  height: 5rem;  
  background-color: lightskyblue;  
  border: solid 1px orange;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  font-size: 3rem;  
  flex-grow: 1;  
}
```

```
.a {  
  order: 1;  
  flex-grow: 2;  
}  
.b {  
  order: -2;  
  flex-grow: 0.5;  
}
```



Flex Shrink

```
.item {  
  width: 15rem;  
  height: 5rem;  
  background-color: lightskyblue;  
  border: solid 1px orange;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  font-size: 3rem;  
}
```

```
.a {  
  order: 1;  
  flex-shrink: 2;  
}  
.b {  
  order: -2;  
  flex-shrink: 0;  
}
```



Flex Basis

```
.item {  
  width: 10rem;  
  height: 5rem;  
  background-color: lightskyblue;  
  border: solid 1px orange;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  font-size: 3rem;  
}
```

```
.a {  
  order: -1;  
  flex-shrink: 2;  
}  
.b {  
  order: 0;  
  flex-basis: 20rem;  
}
```



flex property

- flex là property cho phép thiết lập đồng thời: *flex-grow*, *flex-shrink* và *flex-basis*

```
.flex{  
  flex: 2 0 100px;  
}
```

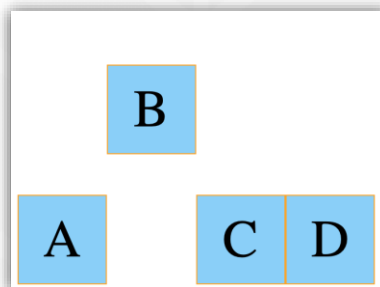


```
.flex{  
  flex-basis: 100px;  
  flex-grow: 2;  
  flex-shrink: 0;  
}
```

Align Self

```
.container{  
  display: flex;  
  align-items: flex-end;  
  height: 20rem;  
}
```

```
.b{  
  order: 0;  
  align-self: center;  
}
```



Nội dung

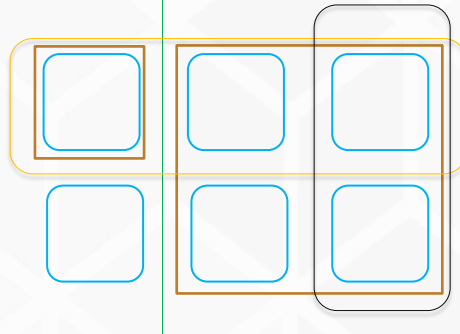
- ☐ *Display*
- ☐ *Flexible box layout*
- ☐ **Grid layout**
- ☐ CSS – Page Layout

Grid layout

- ☐ Bố cục (*layout*) giao diện dựa vào lưới (*grid*) là hướng tiếp cận khả quan nhất. Tuy nhiên với CSS trước giờ thì việc bố cục theo lưới rất khó khăn và phức tạp.
- ☐ Hiện nay **CSS Grid** với mục tiêu cung cấp hệ thống bố cục theo lưới từ gốc giúp cho việc bố cục dễ dàng, đẹp mắt mà không cần sử dụng nhiều **markup** và **style**.

Thuật ngữ

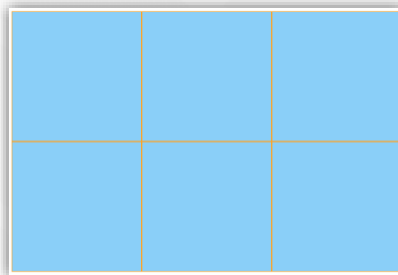
- ❑ Grid Cell
- ❑ Grid Area
- ❑ Grid Row
- ❑ Grid Column
- ❑ Grid Gap
- ❑ Grid Lines



Khai báo grid

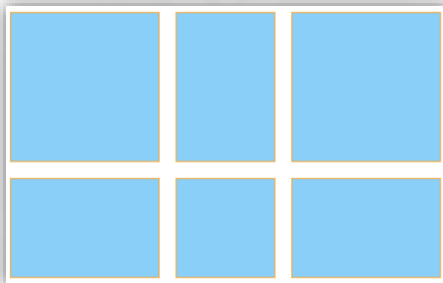
```
.container {  
  display: grid;  
  grid-template-columns: 150px 150px 150px;  
  grid-template-rows: 150px 150px;  
}  
.cell {  
  background-color: lightskyblue;  
  border: solid 1px orange;  
}
```

```
<div class="container">  
  <div class="cell"></div>  
  <div class="cell"></div>  
  <div class="cell"></div>  
  <div class="cell"></div>  
  <div class="cell"></div>  
  <div class="cell"></div>  
</div>
```



Sử dụng gap

```
.container {  
  display: grid;  
  grid-template-columns: 150px 100px 150px;  
  grid-template-rows: 150px 100px;  
  grid-gap: 1rem;  
}
```



FR (Fraction) Unit

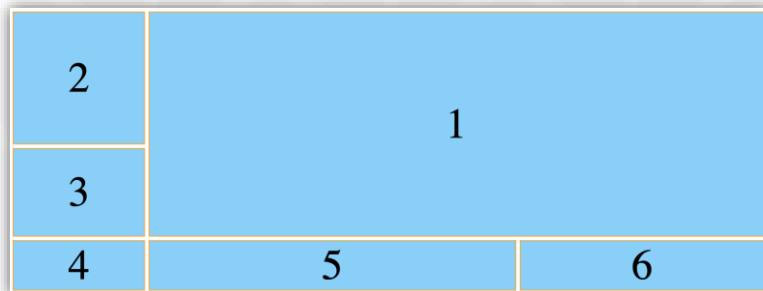
```
.container {  
  display: grid;  
  grid-template-columns: 150px 1.5fr 1fr;  
  grid-template-rows: 150px 100px;  
  grid-gap: 0.25rem;  
}
```



Grid Start - End

```
.s {  
  grid-row: 1 / 3;  
  grid-column: 2 / 4;  
}
```

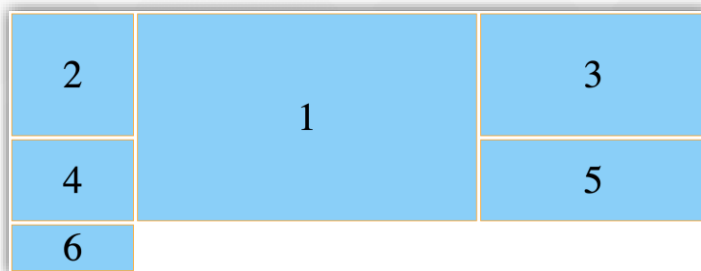
```
.s {  
  grid-row: 1 / span 2;  
  grid-column: 2 / span 2;  
}
```



Named Grid Lines

```
.container {  
  display: grid;  
  grid-template-columns:  
    [column-1] 150px  
    [column-2] 1.5fr  
    [column-3] 1fr;  
  grid-template-rows: 150px 100px;  
  grid-gap: 0.25rem;  
}
```

```
.s {  
  grid-row: 1 / 3;  
  grid-column: column-2 / column-3;  
}
```



Template Areas

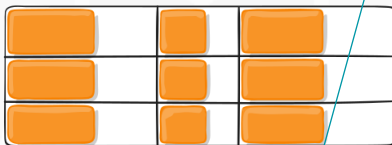
```
.container {  
  display: grid;  
  grid-gap: 0.5rem;  
  grid-template-areas:  
    "top top"  
    "middle-1 middle-2 "  
    "bottom bottom";  
}
```

```
.top {  
  grid-area: top;  
}  
.middle-1 {  
  grid-area: middle-1;  
}  
.middle-2 {  
  grid-area: middle-2;  
}  
.bottom {  
  grid-area: bottom;  
}
```



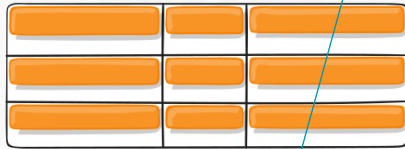
Justify items

```
.container {  
  justify-items: start | end | center | stretch;  
}
```



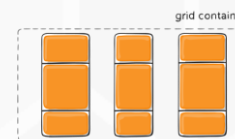
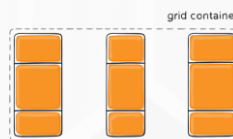
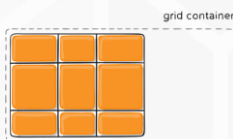
Align items

```
.container {  
  align-items: start | end | center | stretch;  
}
```



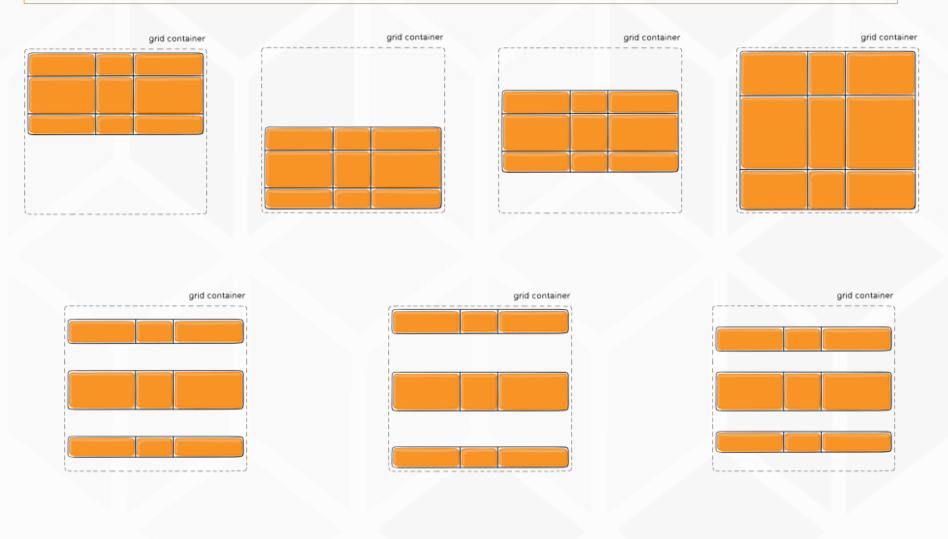
Justify content

```
.container { justify-content: start | end | center | stretch | space-around |  
space-between | space-evenly; }
```



Align content

```
.container { align-content: start | end | center | stretch | space-around | space-between | space-evenly; }
```



Thử thách

- ❑ Flexbox: <https://flexboxfroggy.com>
- ❑ Grid: <https://cssgridgarden.com>

Nội dung

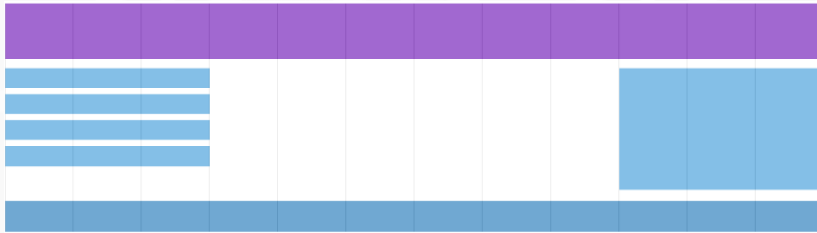
- ☐ *Display*
- ☐ *Flexible box layout*
- ☐ *Grid layout*
- ☐ **CSS – Page Layout**

CSS - Page Layout

- ☐ Trong việc sử dụng CSS thì điều khó khăn nhất là làm sao layout được chính xác các thành phần lên page.
- ☐ Các loại Page layout thông dụng trong CSS:
 - ☐ Two-column Page Layout (sử dụng float)
 - ☐ Sidebar Navigation Layout with Position
 - ☐ Flexbox Layout
 - ☐ Grid Layout

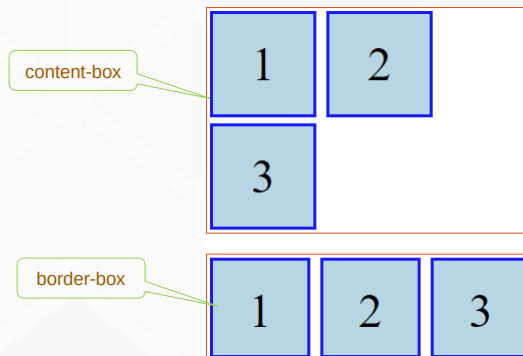
Grid-View

- ❑ Phần lớn các trang web được bố cục trên cơ sở lưới (**grid-view**), có nghĩa là các thành phần được chia vào các cột (**column**), điều này giúp giao diện nhất quán và ràng mạch. **Grid-View** thông dụng có 12 cột với kích thước phù hợp ngữ cảnh.



Box-sizing

- ❑ Mặc định là **content-box** với giá trị **width** xác định chính xác với **content** => khó kiểm soát trong bố cục.
- ❑ Với thiết lập **border-box** giúp xác định giá trị **width** sẽ bao gồm cả **border** và **padding**.



Nội dung

- ☐ Nguyên lý thiết kế web hiện đại
- ☐ CSS – Layout
- ☐ **Responsive web design**

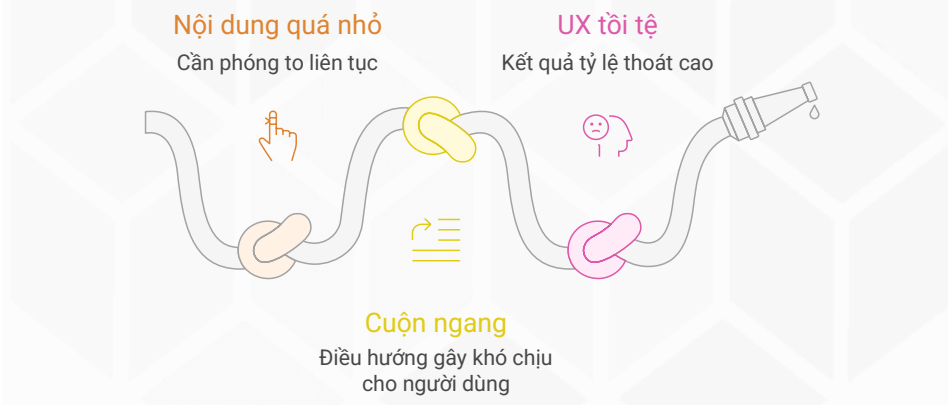
Thiết kế web đáp ứng **Responsive**

- ☐ Website đáp ứng **responsive** là trang web có thể **tự động điều chỉnh** để xem được tốt trên mọi thiết bị: PC, tablet, phone,... (*kích thước màn hình khác nhau*)
- ☐ Website cần đáp ứng **responsive** chỉ với code HTML và CSS, không phải sử dụng Javascript để thực hiện điều này.

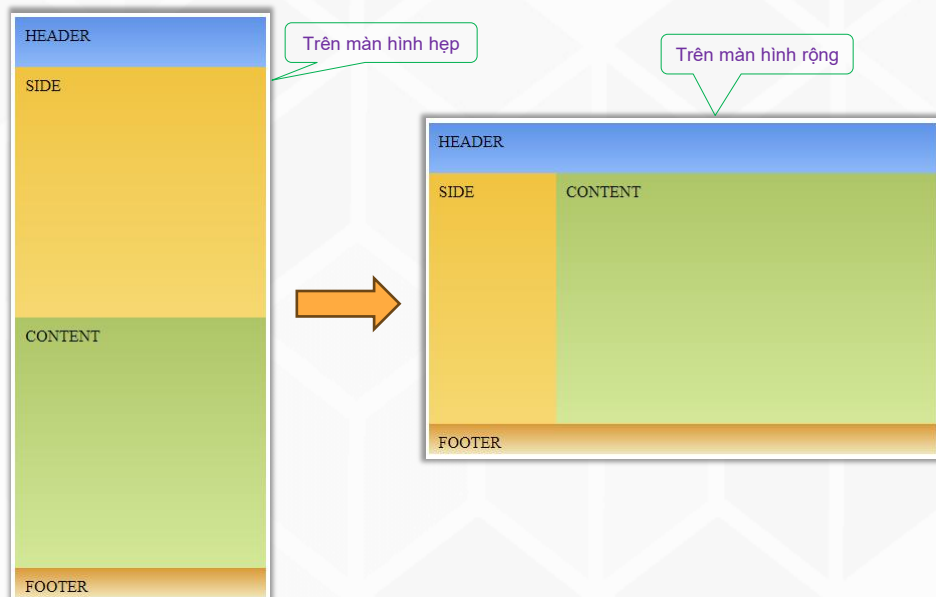


Thế giới đa màn hình

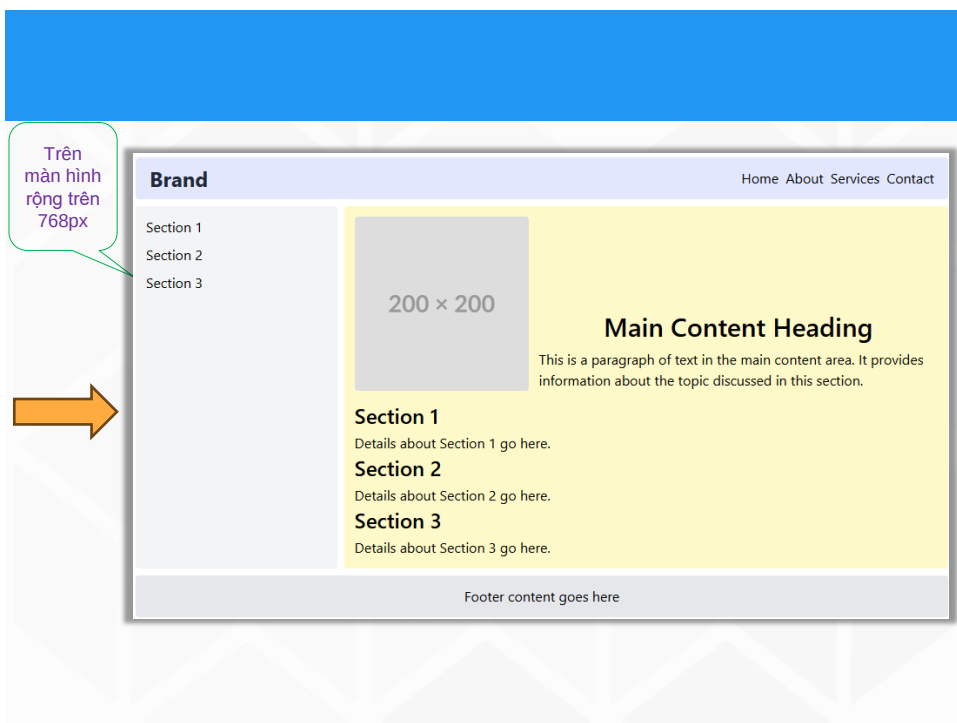
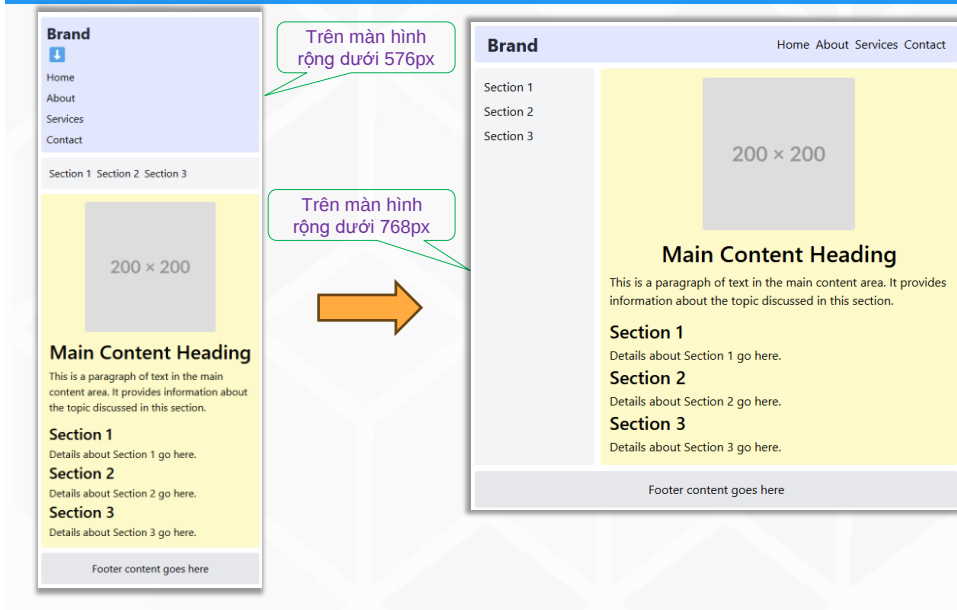
- ❑ **Thực trạng:** Lưu lượng truy cập internet trên di động đã vượt qua máy tính để bàn từ nhiều năm trước.
- ❑ Vấn đề của layout cố định (Fixed Layout):



Ví dụ 1



Ví dụ 2



Ba Trụ cột Kỹ thuật của RWD

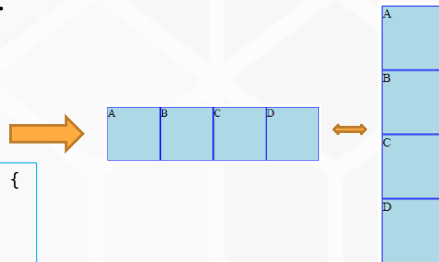
- ❑ **Fluid Grids** (Lưới linh hoạt): Sử dụng đơn vị tương đối như phần trăm (%) thay vì đơn vị cố định (px) cho chiều rộng của các cột layout. Giúp layout co giãn theo kích thước của viewport.
- ❑ **Flexible Images** (Hình ảnh co giãn): Sử dụng thuộc tính `max-width: 100%`; để đảm bảo hình ảnh không bao giờ tràn ra ngoài vùng chứa của nó.
- ❑ **Media Queries** (Truy vấn Media): "Bộ não" của RWD. Cho phép áp dụng các đoạn CSS khác nhau tùy thuộc vào đặc điểm của thiết bị (chủ yếu là chiều rộng màn hình).

Media Queries

- ❑ Media query là một trong những module mới được thêm vào trong **CSS3**. Nó là một sự cải thiện của **Media Type** đã có từ **CSS2**, bằng việc thêm vào những cú pháp **query** giúp ta có thể đáp ứng được cho nhiều **device** với nhiều kích cỡ màn hình khác nhau.

```
.container {  
  display: flex;  
  flex-direction: row;  
}
```

```
@media only screen and (max-width: 600px) {  
  .container {  
    display: flex;  
    flex-direction: column;  
  }  
}
```



Thẻ meta viewport

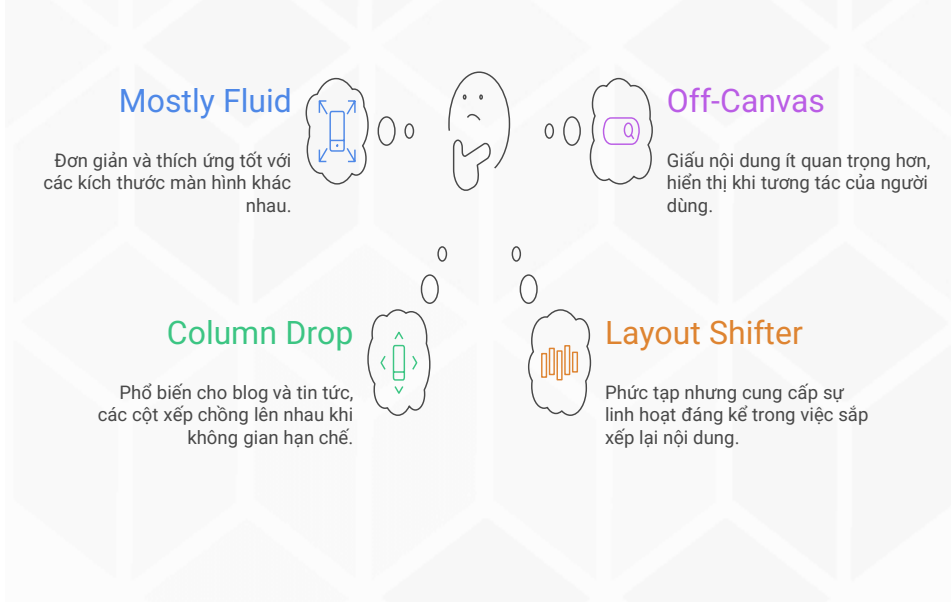
- ❑ Lưu ý đặc thù của trình duyệt trên thiết bị di động thường xử lý trang web với kích thước rộng hơn kích thước thật của thiết bị. Do đó để đáp ứng tốt **responsive** thì trang web cần được thiết lập giá trị **viewport** như sau

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Chiến lược hiện đại: Mobile-First

- ❑ **Desktop-First:** Thiết kế cho màn hình lớn trước, sau đó dùng media query để "sửa lỗi" trên màn hình nhỏ.
 - ❑ *Nhược điểm:* CSS phức tạp, phải ghi đè nhiều, thường tải những tài nguyên không cần thiết trên di động.
- ❑ **Mobile-First:** Thiết kế cho màn hình nhỏ trước, sau đó dùng **media query** (**min-width**) để thêm các yếu tố và sắp xếp lại layout cho màn hình lớn hơn.
- ❑ Ưu điểm:
 - ❑ Tập trung vào nội dung quan trọng nhất.
 - ❑ Tối ưu hiệu năng (performance) cho di động.
 - ❑ Code CSS gọn gàng hơn.

Các Mẫu Layout Responsive Phổ biến



Lưu ý về UX & Performance

- ❑ **Kích thước Touch Target**: Nút bấm, link phải đủ lớn để người dùng có thể chạm bằng ngón tay một cách dễ dàng (tối thiểu 44x44px).
- ❑ **Font chữ**: Đảm bảo font chữ đủ lớn, dễ đọc trên di động.
- ❑ **Navigation**: Sử dụng các mẫu điều hướng phù hợp cho di động (hamburger menu, tab bar).
- ❑ **Tối ưu hình ảnh**: Sử dụng thẻ <picture> hoặc thuộc tính srcset để trình duyệt tải hình ảnh có kích thước phù hợp với màn hình, tránh tải ảnh "siêu to khổng lồ" trên di động.

Công cụ & Kiểm thử

❑ Frameworks:

- ❑ **Bootstrap, Tailwind CSS:** Cung cấp sẵn hệ thống lưới và các lớp tiện ích (utility classes) để xây dựng giao diện responsive nhanh chóng.

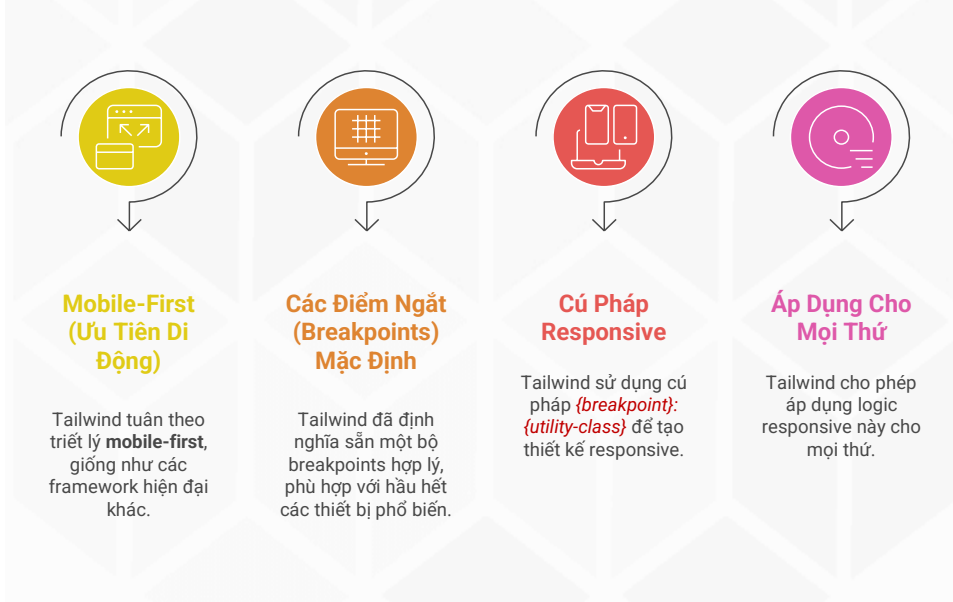
❑ Công cụ kiểm thử quan trọng nhất:

- ❑ **Trình duyệt DevTools (F12):** Chế độ "Device Toolbar" (Ctrl+Shift+M) cho phép giả lập hàng trăm loại thiết bị khác nhau ngay trên máy tính.
- ❑ **Luôn kiểm tra trên thiết bị thật!** Cảm giác sử dụng thực tế luôn khác với giả lập.

RWD – Bootstrap



RWD – Tailwind CSS



Bài tập



- ☐ Làm Ví dụ 1 với CSS đáp ứng **responsive**.
- ☐ Làm Ví dụ 1 với Bootstrap đáp ứng **responsive**.
- ☐ Làm Ví dụ 1 với Tailwind CSS đáp ứng **responsive**
- ☐ Làm Ví dụ 2 với CSS đáp ứng **responsive**
- ☐ Làm Ví dụ 2 với Bootstrap đáp ứng **responsive**
- ☐ Làm Ví dụ 2 với Tailwind CSS đáp ứng **responsive**